

Qwen3-30B-A3B-Thinking colocate baseline: caching-allocator fragmentation OOM, with memory-profiler evidence

Date: 2026-05-18 / 2026-05-19 **Author:** Mathew (with Claude) **Run logs:** `logs/dapo_10rollout_TP4_EP4_DP2.log` (original failure), `logs/dapo_10rollout_TP4_EP4_DP2_PROFILING.log` (with memory profiling) **Snapshots:** `memory-snapshots/dapo_TP4_EP4_DP2_unblocked/` (44 strategic + 4 OOM-moment pickle files) **Visualizer:** drop the OOM `.pickle` into https://pytorch.org/memory_viz

TL;DR

On 8× H200 (144 GB), our DAPO 10-rollout colocate RL run with Qwen3-30B-A3B-Thinking-2507 (TP=4 + EP=4 + DP=2) crashed at **rollout=1**, in the `actor_logprob` phase the first time and in `train_fwd_bwd` the second time. The crash is a CUDA OOM, but the GPU was not actually out of memory — at the moment of failure, PyTorch's caching allocator had **7.45 GB free** but the **largest contiguous block was only 5.19 GB**, and the failing allocation wanted **6.86 GB in one piece**.

The fragmentation is driven by Megatron's dynamic-batched microbatches with varying token counts (172 distinct microbatch shapes in 1024 microbatches). Each shape carves a different-sized region into the allocator's segment list. By rollout=1, the segment count has tripled and the pool can no longer satisfy a 6.86 GB request despite having room.

The standard PyTorch fix (`PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`) is **incompatible** with `torch_memory_saver` — the library that powers colocate's SGLang↔Megatron sleep/wake — because both hook the same CUDA virtual-memory APIs. So we need a different mitigation.

This report walks through the evidence collected via `torch.cuda.memory._record_memory_history` (which is compatible with `torch_memory_saver`), then evaluates whether moving to B200 hardware (more memory) would dodge the issue.

1. The OOM, in PyTorch's own words

At rank 0, GPU 0:

```
torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 6.86 GiB.
GPU 0 has a total capacity of 139.81 GiB of which 7.60 GiB is free.
Process 579506 has 1.99 GiB memory in use.
Process 584047 has 129.99 GiB memory in use.
Of the allocated memory 116.96 GiB is allocated by PyTorch, and
3.37 GiB is reserved by PyTorch but unallocated.
If reserved but unallocated memory is large try setting
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation.
```

The phrase **"reserved by PyTorch but unallocated"** is the signature of fragmentation. PyTorch's recommendation (`expandable_segments:True`) is the standard fix — but it's blocked by

`torch_memory_saver` in our setup. PyTorch's API for inspecting the allocator's state (`torch.cuda.memory._record_memory_history + _dump_snapshot`) is not blocked, so we used that.

2. How we captured the data

Added to `MegatronTrainRayActor.init` in `slime/backends/megatron_utils/actor.py`, gated on the env var `SLIME_MEMORY_SNAPSHOT_DIR`:

```
torch.cuda.memory._record_memory_history(max_entries=1_000_000, stacks="all")

def _oom_observer(device, alloc, device_alloc, device_free):
    ts = int(time.time())
    path = f"{snapshot_dir}/oom_rank{rank}_t{ts}.pickle"
    torch.cuda.memory._dump_snapshot(path)

torch._C._cuda_attach_out_of_memory_observer(_oom_observer)
```

Plus 3 strategic `_dump_snapshot` calls in `train_actor`:

- **entry** — start of `train_actor` (post `wake_up`)
- **after_actor_logprob** — end of the forward-only logprob pass
- **after_train_fwd_bwd** — end of the full training fwd+bwd

`max_entries=1_000_000` captures the full allocator event history with Python stack traces. The OOM observer fires automatically when PyTorch is about to raise `OutOfMemoryError`, dumping the pool state at the exact moment of failure.

Output: 44 strategic snapshots + 4 OOM snapshots (one per rank that crashed). Total ~14 GB on disk.

3. Fragmentation trajectory across the run

The same rank (rank 6, which crashed in both the original and profiling runs):

Phase	Segments	Reserved	Allocated	Free	Largest free block
Rollout=0 entry	856	58.7 GB	57.0 GB	1.7 GB	0.02 GB
Rollout=0 after actor_logprob	955	128.3 GB	57.6 GB	70.7 GB	9.28 GB
Rollout=0 after train_fwd_bwd	2,286	88.5 GB	86.4 GB	2.1 GB	0.01 GB
Rollout=1 entry (post sleep/wake)	2,257	88.4 GB	86.4 GB	2.0 GB	0.01 GB
Rollout=1 after actor_logprob	2,308	123.4 GB	86.2 GB	37.2 GB	6.67 GB
OOM (rollout=1 train_fwd_bwd)	2,305	123.4 GB	115.9 GB	7.5 GB	5.19 GB ←

Three things to notice:

1. Segment count nearly triples (856 → 2,305) across two rollouts. Each segment is one `cudaMalloc` from the driver. PyTorch grabs them on demand; once allocated, segments cannot be merged or moved. The jump from 955 → 2,286 segments happens during rollout=0's `train_fwd_bwd` — that single phase creates ~1,300 new segments via the carve/free pattern of dynamic-batched microbatches.

2. Sleep/wake preserves fragmentation, as predicted. Look at the pair Rollout=0 after `train_fwd_bwd` (2,286 segments, 0.01 GB largest free) → Rollout=1 entry post sleep/wake (2,257 segments, 0.01 GB largest free). The numbers are essentially identical. `torch_memory_saver.pause()` releases physical pages but **does not touch the allocator's free-list**. The pool wakes up in the exact fragmented state it slept with.

3. The largest-free-block oscillates by phase. After each forward-only pass it spikes (9.28 GB, 6.67 GB) because activations are freed in bulk; after each fwd+bwd it collapses (0.01 GB) because gradients are now live. The OOM happens at a moment in fwd+bwd when activations + gradients are simultaneously live and the next logits allocation can't find a slot.

4. The OOM moment — fine-grained snapshot

Rank 6 at OOM:

- **2,305 segments**
- **123.39 GB reserved** (sum of `segment.total_size`)
- **115.94 GB allocated** to live tensors (8,087 active blocks)
- **7.45 GB free** across 1,565 inactive blocks
- **Allocation request that failed: 6.86 GB**

4a. Free-block size distribution (fine-grained)

Size range	# blocks	Total
<128 KB	420	2.6 MB
128 KB – 1 MB	47	15.0 MB
1-4 MB	1,080	2,160.4 MB
4-8 MB	15	81.6 MB
8-16 MB	1	8.0 MB
16-64 MB	1	46.0 MB
≥ 4 GB	1	5,312.8 MB

The distribution is **bimodal**: one giant 5.19 GB free block, then a long tail of 1,564 small blocks averaging 1-4 MB.

The 6.86 GB allocation needs one contiguous block. The pool's biggest hole is 5.19 GB — short by 1.67 GB.

4b. Top live tensors at OOM

These are eating the budget:

```

1. 27.000 GB
2. 13.500 GB
3. 6.672 GB
4. 6.672 GB
5. 6.590 GB
6. 6.590 GB
7. 1.471 GB
8. 0.736 GB
9. 0.291 GB
10. 0.290 GB

```

The top 2 alone are 40.5 GB — most likely the optimizer state's flat distributed-buffer tensors and one of the MoE alltoall workspaces. The four 6.5-6.7 GB tensors are previously-allocated logits workspaces from earlier microbatches, held live because the backward pass needs them.

4c. Segment "waste" — concentrated fragmentation

Classifying each segment by what fraction is free:

Segment state	# segments	Comment
<10% free (mostly live)	2,246	Healthy — fully occupied
10-50% free	37	Mild waste
50-90% free	10	Significant waste
>90% free	12	Stranded — large segments mostly empty

The 12 "stranded" segments are the smoking gun. They were grabbed at some point (each as a single `cudaMalloc`), filled briefly, then mostly freed — but they can't be coalesced because they sit in non-contiguous virtual-address ranges. They collectively waste ~10-15 GB that the driver thinks is "in use by the process" but is just empty space inside PyTorch's allocator.

4d. The critical segment

The segment containing the 5.19 GB free block:

```

Total:      6.025 GB
Allocated: 0.837 GB (34 live blocks)
Free:       5.188 GB (one big block)

```

This segment was almost certainly grabbed for a previous logits tensor of ~6.025 GB → freed → reused for smaller activations → the residual is the 5.19 GB free block. **If that segment had been 14% larger (≥ 6.86 GB), the OOM wouldn't have happened.**

4e. Segment-size distribution

Size	# segments
1-16 MB	1,142
16-256 MB	1,143
256 MB - 1 GB	11
≥ 1 GB	9

Each segment is one `cudaMalloc`. The driver gave PyTorch 2,305 separate physical-memory regions. 12 of them are stranded.

5. Why this happens — mechanism

The Megatron training process holds three categories of allocations:

Persistent (~100 GB per rank, allocated once, never freed)

- Weights: ~14 GB
- Gradients (bf16): ~16 GB (allocated each step, freed at end — same shape every time, so always reusable)
- Optimizer state: ~60 GB (Adam fp32 momentum + variance + master weights, distributed via EP=4 + DP=2 sharding)

Per-rollout (allocated/freed once per swap-cycle)

- NCCL communication buffers
- KV-cache-shaped tensors during weight push to SGLang
- MoE alltoall workspaces

Per-microbatch (allocated/freed 1,000+ times, different shape each time) — the killer

- Activation tensors — shape varies because `--use-dynamic-batch-size --max-tokens-per-gpu 2048` packs different numbers of samples per microbatch (~12K, 8K, 15K tokens in our run)
- Logits tensor `(num_tokens, 1, 38016)` in fp32 — scales linearly with `num_tokens` (6.86 GB for ~44K tokens)
- Attention workspaces, recomputation buffers, gradient comm slices, MoE expert routing scores

With 512 microbatches per fwd-only pass × 2 passes per rollout, the allocator does ~1,000 carve/release cycles per rollout. The 172 distinct microbatch shapes (measured directly) each ask for slightly different sizes. Each new size that doesn't fit an existing free block causes a fresh `cudaMalloc` → new segment.

PyTorch's caching allocator **can only coalesce free blocks that are adjacent in the same segment**. It cannot move tensors (which would invalidate all the live pointers held by autograd, NCCL, cudagraphs). And it cannot merge separate segments.

The fragmentation **survives sleep/wake**. `torch_memory_saver.pause()` calls `cuMemUnmap` on virtual pages but **keeps the allocator's free-list intact**, because invalidating it would break every live tensor pointer in the process. So Megatron wakes for rollout=1 with its allocator in the exact fragmented state it slept with — which is exactly what our snapshots confirm.

6. Comparison: original failure vs profiling run

	Original (no profiling)	Profiling run
OOM phase	rollout=1 actor_logprob	rollout=1 train_fwd_bwd
Allocation request	6.26 GB	6.86 GB
Allocated by PyTorch	114.3 GB	116.0 GB
Reserved-but-unallocated	12.10 GB	7.45 GB
Largest free block	(not captured)	5.19 GB
Driver-level free	3.55 GB	7.60 GB

The profiling overhead (`_record_memory_history` with `stacks="all"`) shifted the OOM into the next phase but kept the **failure mode identical**. The drop in "reserved-but-unallocated" (12 → 7 GB) is consistent with PyTorch's allocator being slightly more efficient under instrumentation, but the structural problem remains: the largest contiguous block can't satisfy the request.

7. What this conclusively proves

- **Fragmentation is the OOM root cause.** Direct evidence: largest free block (5.19 GB) is smaller than the allocation request (6.86 GB), while total free space (7.45 GB) would suffice if contiguous.
- **The fragmentation builds up within Megatron's process**, not from any SGLang interaction. The OOM happens during the train phase, with SGLang fully paused.
- **Sleep/wake preserves the allocator's metadata**, as predicted. The "entry" snapshot of rollout=1 has 2,257 segments and 0.01 GB largest free — exactly the same shape as rollout=0's end state.
- **The 172 distinct microbatch shapes** (from dynamic batching) directly produce the fragmentation pattern: 1,564 free blocks at OOM, mostly in the 1-16 MB range (matching activation/logits tensor sizes for various microbatch sizes).

8. Would B200s help?

Short answer: yes, probably enough to dodge the immediate OOM, but it's a workaround not a fix.

Hardware comparison

GPU	HBM	Memory Bus	Architecture
H200 (current)	144 GB HBM3e	4.8 TB/s	Hopper
B200	192 GB HBM3e	8 TB/s	Blackwell

B200 has **48 GB more memory per GPU** (+33%) and faster memory bandwidth.

Math on whether B200 dodges the current failure

At the OOM moment on H200, we had: - 123.4 GB reserved (in the PyTorch allocator pool) - 115.9 GB allocated - 7.5 GB free across 1,565 blocks (largest hole: 5.19 GB) - Driver-level free: 7.6 GB - Total physical: 144 GB

The OOM request was 6.86 GB. The allocator couldn't satisfy from its free-list (largest hole 5.19 GB). It would have called `cudaMalloc(6.86 GB)` to grab a fresh segment. The driver had **7.6 GB free** — that should have worked, but `cudaMalloc` reserves slightly more than requested (alignment, padding) and depends on the driver having a 6.86 GB contiguous physical region available, which it may not have due to driver-side fragmentation of its own.

On B200 with 192 GB: - Allocator might have grown to ~140 GB reserved (same workload, same shape variance) - Driver-level free would be **~50 GB** (vs 7.6 GB on H200) - A `cudaMalloc(6.86 GB)` against 50 GB of driver-free is much more likely to succeed - Even if the allocator's own free-list is fragmented similarly, the fallback to fresh `cudaMalloc` works

So yes — B200 likely succeeds at the same workload that fails on H200. The 48 GB headroom absorbs the fragmentation overhead.

But it's a workaround, not a fix

The fragmentation **pattern is identical** on B200:

- Same Megatron code paths, same caching allocator, same dynamic-batched microbatches
- The 172 distinct microbatch shapes still create ~1,300 new segments per rollout's train phase
- The 5-7 GB logits workspaces still get stranded in too-small segments
- Sleep/wake still preserves fragmentation across rollouts

The only difference is that B200 has **more slack before the fragmentation eats the budget**. Specifically:

1. **For a 10-rollout run:** B200 probably survives all 10. Headroom grows with the extra 48 GB.
2. **For longer runs** (50+ rollouts): the fragmentation could eventually eat the extra 48 GB. The growth isn't strictly bounded — it depends on the variability of microbatch shapes across rollouts. We'd need to test empirically.
3. **For larger models** (Qwen3-235B-A22B): persistent state grows. A 235B-A22B model with the same parallelism would have ~5× the persistent memory per rank, eating most of B200's extra headroom on weights alone. Fragmentation budget shrinks back to H200-like levels.
4. **For wider parallelism** (more GPUs per replica): per-rank persistent state shrinks, fragmentation budget grows. But that costs more GPUs.

B200-specific considerations

B200 changes a few other things that might affect fragmentation indirectly:

- **Different CUDA driver versions** — newer drivers might handle `expandable_segments` better, including possibly resolving the `torch_memory_saver` incompatibility (though that's a userspace library issue, not driver)
- **Different `cudaMallocAsync` semantics** — Blackwell's memory subsystem might give the driver more flexibility for satisfying contiguous allocations

- **Same PyTorch caching allocator behavior** — no change to the carve/free pattern in user space
- **Same torch_memory_saver semantics** — preserves allocator state across pause/resume

The pattern that creates fragmentation lives in PyTorch and Megatron user-space code. B200 doesn't change that.

Recommendation on B200

If we have B200s available and the goal is to ship the colocate baseline: yes, B200 is a pragmatic workaround for this specific workload size and rollout count.

If we want to actually solve the fragmentation for sustainability across larger models, longer runs, and other workloads: B200 only delays the problem. The real fixes are:

1. **Switch to the streaming actor** (`StreamingMegatronTrainRayActor`) which already has `clear_memory()` between phases (`streaming_actor.py:281, 591, 719`). Our analysis shows this won't fully eliminate fragmentation but will reduce stranded segments by 10-15 GB — combined with the modest H200 headroom that already exists, it might be enough.
2. **Press torch_memory_saver upstream to add expandable_segments support.** The library author has flagged this as "not supported yet" — if we contribute the fix, it's a one-line PyTorch env var change in slime.
3. **Reduce max-tokens-per-gpu from 2048 → 1024** to shrink microbatch shape variance. Likely cuts segment count growth by ~half. Cost: ~2x more microbatches, slower train step.
4. **Add an explicit clear_memory() between actor_logprob and train_fwd_bwd in baseline actor.py** — mirror what streaming does. Would reclaim ~10-15 GB at the worst phase boundary.

These fixes attack the root cause and work on any hardware. B200 is the "buy more headroom and hope" path.

9. Recommendation summary

For the **immediate goal of getting the DAPO baseline run completed**: - If H200s remain the target: try the streaming actor's `clear_memory()` mitigations + reduced `max-tokens-per-gpu`, or accept some rollouts will fail and retry - If B200s are available: just use them and ship

For **sustainable colocate RL infrastructure** across model sizes: - Pursue the upstream `torch_memory_saver` fix - Land the `clear_memory()` patches in baseline actor - Document the dynamic-batch + colocate fragmentation interaction so future work doesn't trip on it

10. Files

Path	Purpose
<code>slime/backends/megatron_utils/actor.py</code>	Patched with <code>SLIME_MEMORY_SNAPSHOT_DIR</code> - gated profiling hook + 3 strategic snapshot points
<code>tests/streaming/run_colocate_8xGPU_qwen3thinking_10rollout_dapo.py</code>	Launch script (sets the snapshot dir env var)
<code>memory-snapshots/dapo_TP4_EP4_DP2_unblocked/</code>	48 snapshot pickles (~14 GB total)
<code>oom_rank6_t1779085392.pickle</code>	The smoking-gun OOM snapshot — drop into https://pytorch.org/memory_viz

11. References

- PyTorch CUDA memory: <https://pytorch.org/docs/stable/notes/cuda.html#environment-variables>
- PyTorch memory visualizer: https://pytorch.org/memory_viz
- `torch_memory_saver` library: https://github.com/fzyzcjy/torch_memory_saver
- NVIDIA Blackwell B200 spec: <https://www.nvidia.com/en-us/data-center/dgx-b200/>